

第十章 输入输出流

主讲： 刘晓光 张海威
张 莹 殷爱茹
沈 玮 宋春瑶
李雨森





C++流类库



插入与提取运算符重载



输入/输出格式控制



磁盘文件的输入与输出



文本文件与二进制文件



字符串流



C++流类库



插入与提取运算符重载



输入/输出格式控制



磁盘文件的输入与输出



文本文件与二进制文件



字符串流

流类库的特点

用C++语言自己的支持I/O 操作的流类库代替
printf 函数族，是一个明显的进步

- C语言的printf函数族
 - 由一族函数，控制输入输出，例如：
 - printf()
 - scanf()
- C++语言的流类库
 - 将输入输出封装成流类，调用相关成员函数控制输入输出，例如：
 - `cout<<a`
 - `cin>>b`
 - `cout.width(10)`

流类库的特点

C++语言的输入输出操作(功能)是由它所预定义的输入/输出流类库所提供的

- 简明与可读性
 - I/O 语句更为简明，增加了可读性
- 类型安全（type safe）
 - 所谓类型安全，是指在进行I/O 操作时不应对于参加输入输出的数据在类型上发生不应有的变化
- 易于扩充

文件与流的概念

流 (Stream)

- 流是一个逻辑概念
- 是C++语言对所有外部设备的逻辑抽象
- 代表的是某种流类类型的一个对象
- C++的I/O系统将每个外部设备都转换成一个称为流的逻辑设备，由流来完成对不同设备的具体操作

文件 (File)

- 文件(File)是一个物理概念，代表存储着信息集合的某个外部介质，它是C++语言对具体设备的抽象，如，磁盘文件，显示器，键盘。又可以分为文本和二进制文件

文件与流的行为

所有流(类对象)的行为都是相同的，而不同的文件则可能具有不同的行为

- 磁盘文件可进行写也可进行读操作
- 显示器文件则只可进行写操作
- 键盘文件只可进行读操作

文件与流的联系

- 当程序与一个文件交换信息时，必须通过“打开文件”的操作将一个文件与一个流(类对象)联系起来。
- 文件与流建立联系后，对该流(类对象)的访问就是对该文件的访问，也就是对一个具体设备的访问。
- 可通过“关闭文件”的操作将一个文件与流(类对象)的联系断开。

基本流类

- **ios**
 - 基本流类的基类;
- **istream**
 - 由**ios**派生, 支持输入 (提取 “>>”) 操作;
- **ostream**
 - 由**ios**派生, 支持输出 (插入 “<<”) 操作;
- **iostream**
 - 由**istream**与**ostream**共同派生, 支持输入和输出双向操作。

预定义的流类对象

- **extern istream cin;**
 - 对象cin对应于键盘文件
- **extern ostream cout;**
 - 对象cout对应于显示器文件
- **extern ostream cerr;**
 - 对象cerr对应于显示器文件
- **extern ostream clog;**
 - 对象clog对应于显示器文件
- 流类对象与文件之间的联系已经预定义
- 可认为系统已为每一程序都隐含进行了对它们的打开与关闭操作
- 程序中可直接对这4个预定义流类对象进行读写，而不必先进行“打开文件”的操作，使用完后也不需要进行“关闭文件”的操作

用于磁盘操作的文件流类

在头文件<fstream>中定义

- **ifstream**
 - 由**istream**派生，支持从磁盘文件中输入(读)数据；
- **ofstream**
 - 由**ostream**派生，支持往磁盘文件中输出(写)数据；
- **fstream**
 - 由**iostream**派生，支持对磁盘文件进行输入和输出数据的双向操作。

自定义文件流类对象

C++中没有预定义的文件流类的对象，程序中用到的所有文件流类对象都要自定义

- 自定义ifstream对象进行读文件操作

```
int x;  
ifstream infile("myfile.txt");  
infile>>x;
```

- 自定义ofstream对象进行写文件操作

```
ofstream outfile;  
outfile.open("myfile.txt");  
outfile<<"Write to file";
```

C++流类库 ☐
插入与提取运算符重载 ☒
输入/输出格式控制 ☐
磁盘文件的输入与输出 ☐

☐ 插入与提取运算符重载
☐ 程序示例



C++流类库



插入与提取运算符重载



输入/输出格式控制



磁盘文件的输入与输出



文本文件与二进制文件



字符串流

插入与提取运算符

插入与提取运算符（<<和>>）只能实现基本数据类型☐的输入输出操作

- 由C++预先定义此功能
- 以运算符重载的方式实现
 - 使用流类对象cout输出整型数据x：
`cout<<x;`
相当于
`cout.operator<<(x)`
– cout为对象名，`operator<<`相当于函数名

插入与提取运算符重载

插入与提取运算符（<<和>>）无法直接实现用户自定义的类对象进行输入输出操作

- **【例如】** 自定义一个complex类描述复数

```
complex cp;
```

```
cout<<cp<<endl; //无法实现复数输出功能
```

- 需要在类complex的定义中，对运算符<<进行重载，使其实现输出复数的功能
- 提取运算符>>同样需要进行重载实现输入功能

【例10.1】

重载插入与提取运算符，实现复数的输入和输出

```
#include<fstream>
using namespace std;
class complex {
    double r;
    double i;
public:
    complex(double r0=0, double i0=0) {
        r=r0;i=i0;
    }
    complex operator +(complex c2);
    complex operator *(complex c2);
    friend istream& operator>>(istream& in, complex& com);
    friend ostream& operator<<(ostream& out,complex com);
};
```

```
complex complex::operator +(complex c2) {  
    complex c;  
    c.r=r+c2.r;  
    c.i=i+c2.i;  
    return c;  
}  
complex complex::operator * (complex c2) {  
    complex temp;  
    temp.r=(r*c2.r)-(i*c2.i);  
    temp.i=(r*c2.i)+(i*c2.r);  
    return temp;  
}  
istream& operator >> (istream& in, complex& com) {  
    in>>com.r>>com.i;  
    return in;    //不可缺少，因为函数返回类型为“istream&”  
}
```

```
ostream& operator << (ostream& out, complex com) {  
    out<<" ("<<com.r<<" , "<<com.i<<") "<<endl;  
    return out; //不可缺少，因为函数返回类型为“ostream&”  
}
```

```
void main() {  
    complex c1(1,1), c2(2,3), c3, res;  
    cout<<"c1="<<c1<<"c2="<<c2;  
    res = c1+c2;  
    cout<<"c1+c2="<<res;  
    cout<<"c1*c2="<<c1*c2;  
    cout<<"Input c3:";  
    cin>>c3;  
    cout<<"c3+c3="<<c3+c3;  
}
```

/*注意输入输出语句中出现的类对象`cout`以及`cin`正是输入输出重载函数中引用型形参`out`以及`in`的对应实参。即是说，若使用“`cout<<c1;`”它将等同于调用运算符重载函数“`operator<<(cout, c1);`”，而使用“`cin>>c3;`”则等同于调用函数“`operator>>(cin, c3);`”*/

程序执行后，屏幕显示结果为：

`c1=(1, 1)`

`c2=(2, 3)`

`c1+c2=(3, 4)`

`c1*c2=(-1, 5)`

Input c3: **3 -5**

`c3+c3=(6, -10)`

C++流类库 ☐
插入与提取运算符重载 ☐
输入/输出格式控制 ☒
磁盘文件的输入与输出 ☐

☐ 格式控制函数
☐ 格式控制符
☐ 格式控制程序示例



C++流类库



插入与提取运算符重载



输入/输出格式控制



磁盘文件的输入与输出



文本文件与二进制文件



字符串流

ios类的公有成员函数

- 在ios类中定义了一批公有的格式控制标志位以及一些成员函数
 - 先用某些成员函数来设置标志位
 - 再使用另一些成员函数来进行格式输出。
- ios类中还设置了一个long型的数据成员用来记录当前被设置的格式状态，该数据成员被称为格式控制标志字(或标志状态字)。标志字是由格式控制标志位来“合成”的。
- 注意，ios类作为诸多I/O流类的基类，其公有成员函数能够被各派生类的对象所直接调用

格式控制标志字

每一枚枚举常量值都代表着格式控制标志字中的某一个二进制位(bit)，当设置了某个标志位属性时，该位将取值“1”，否则该位取值“0”（详见P330表10.1）

- 格式控制标志字设置函数
 - setf()
 - ...
- 格式控制标志字重置函数
 - unsetf()

格式控制标志字

通过使用位运算符“|”将多个格式控制标志位属性进行“合成”。但从使用角度看，所设置的标志位属性不能产生互斥。

- 格式控制标志字中设立了三组平行的标志位，程序员应保障任何时刻只设置其中的某一个标志位
 - 表示数制标志位的ios::dec、ios::oct和ios::hex
 - 表示对齐标志位的ios::left、ios::right和ios::internal
 - 表示实数格式标志位的ios::scientific和ios::fixed

ios::flags

long flags(long IFlags);

- 通过参数IFlags来重新设置标志字并返回
- 表示各标志位的枚举常量，即IFlags的取值包括(参看P332-P333)
 - ios::skipws = 0x0001，即0000000...0001
 - ios::left
 - ios::right
 - ...
 - ios::stdio

long flags();

- 返回当前的标志字

ios::setf

long setf(long IFlags);

- 通过参数IFlags来设置指定的格式控制标志位。
 - 注意，与flags函数的“替换”方式不同，此处为“添加”方式，即是说，它并不更改其它IFlags不涉及到的那些标志位的当前值。

long setf(long IFlags, long IMasks);

- 设置指定的格式控制标志位的值
 - 首先将第二参数IMask所指定的那些位清零
 - 而后用第一参数IFlags所给定的值来重置这些标志位

ios::setf

设置数制标志位时不产生互斥，例如设置16进制

- `setf(ios::hex, ios::basefield);`
 - `ios::basefield`为一个在`ios`类中定义的公有静态常量，它的取值为`ios::dec|ios::oct|ios::hex`

设置对齐标志位时不产生互斥，例如设置左对齐

- `setf(ios::right, ios::adjustfield);`
 - `ios::adjustfield`为一个在`ios`类中定义的公有静态常量，它的取值为`ios::left|ios::right|ios::internal`

设置实数格式标志位时不产生互斥，例如设置定点格式

- `setf(ios::fixed, ios::floatfield);`
 - `ios::floatfield`为一个在`ios`类中定义的公有静态常量，它的取值为`ios::scientific|ios::fixed`

其它格式控制函数

ios::unsetf

- `long unsetf( long lFlags)`
- 通过参数lFlags来清除指定的格式控制标志位。

ios::fill

- `char fill( char cFill)`
- 将“填充字符”设置为cFill, 并返回原“填充字符”

其它格式控制函数

ios::precision

- `int precision( int np)`
- 设置浮点数精度为np并返回原精度。
- 当格式为ios::scientific或ios::fixed时，精度np指小数点后的位数，否则指有效数字

ios::width

- `int width( int nw)`
- 设置当前被显示数据的域宽nw并返回原域宽。
- 默认值为0，将按实际需要的域宽进行输出。
- 此设置只对随后的一个数据有效，而后系统立刻恢复域宽为系统默认值0

格式控制符

可直接用于提取和插入算符(“>>”和“<<”)之后，而不像格式控制成员函数那样须被单独调用

- `iostream.h`中定义的
无参格式控制符

- `endl`
- `ends`
- `flush`
- `ws`
- `dec`
- `hex`
- `oct`

- `iomanip.h`中定义的
有参格式控制符

- `setbase(int base)`
- `resetiosflags(long lFlags)`
- `setiosflags(long lFlags)`
- `setfill(char cFill)`
- `setprecision(int np)`
- `setw(int nw)`

【例10.2】

- 输入输出格式控制函数的使用

```
#include <iostream>
using namespace std;
void main() {
    cout<<ios::basefield;    //输出: 3584
    cout<<" "<<(ios::dec|ios::oct|ios::hex)<<endl;
    //输出: 3584, 即三个格式控制字进行按位异或运算的值
    cout<<ios::adjustfield; //输出: 448
    cout<<" "<<(ios::left|ios::right|ios::internal);
    //输出: 448
    cout<<endl;
    cout<<ios::floatfield;   //输出: 12288
    cout<<" "<<(ios::scientific|ios::fixed)<<endl;
```


//flags将重新设置标志字, “替换”方式

```
cout.flags (ios::showbase) ;  
cout<<cout.flags ()<<endl; //输出: 8  
cout.flags (ios::showpoint) ;  
cout<<cout.flags ()<<endl; //输出: 16  
cout.unsetf (ios::showpoint) ;  
cout<<cout.flags ()<<endl; //输出: 0
```

//setf为“添加”方式

```
cout.setf (ios::showbase) ;  
cout<<cout.flags ()<<endl; //输出: 8  
cout.setf (ios::showpoint) ;  
cout<<cout.flags ()<<endl; //输出: 24  
cout.unsetf (ios::showpoint) ;  
cout<<cout.flags ()<<endl; //输出: 8
```

```
}
```

【例10.3】

- 输入输出格式控制函数与格式控制符的使用

```
#include <iomanip.h>
void main() {
    cout.width(6); //只管随后一个数的域宽
    cout<<4785<<27.4272<<endl; // 478527.4272
    cout<<setw(6)<<4785<<setw(8)<<27.4272<<endl;
    // 4785 27.4272
    cout.width(6);
    cout.precision(3);
    /*当格式为ios::scientific或ios::fixed时，浮点数精度
    np指小数点后的位数，否则指有效数字，此时设置浮点数的有效数
    字为3*/
    cout<<4785<<setw(8)<<27.4272<<endl;
    //4785      27.4
```

```
cout<<setw(6)<<4785<<setw(8)<<setprecision(2);  
cout<<27.4272<<endl<<endl;  
//  4785      27  
//setprecision(2) 设置浮点数的有效数字  
cout.setf(ios::fixed, ios::floatfield);  
//今后以定点格式显示浮点数(无指数部分)  
cout.width(6);  
cout.precision(3);  
//当格式为ios::fixed时, 设置小数点后的位数  
cout<<4785<<setw(8)<<27.4272<<endl;  
//  4785    27.427  
}
```

C++流类库 ☐
插入与提取运算符重载 ☐
输入/输出格式控制 ☐
磁盘文件的输入与输出 ☒

☐ 文件的打开与关闭
☐ 使用插入与提取运算符进行文件读写
☐ 使用成员函数对文件流类对象进行操作



C++流类库



插入与提取运算符重载



输入/输出格式控制



磁盘文件的输入与输出



文本文件与二进制文件



字符串流

读写磁盘文件的一般处理过程

打开文件

读写操作

关闭文件

- “打开文件” 通常通过构造函数自动完成（也可显式调用成员函数open完成）
- “关闭文件” 通常通过使用 “<流类对象名>.close();” 来显式完成。

打开文件

```
ofstream outfile1("myfile1.txt");
```

- 创建ofstream类的对象outfile1；使流类对象outfile1与磁盘文件"myfile1.txt"相联系；并打开用于“写”的磁盘文件"myfile1.txt"。

```
ofstream outfile1;
```

- 创建ofstream类的对象outfile1

```
outfile1.open("myfile1.txt");
```

- 显式调用成员函数open来打开文件

文件流类的构造函数

```
ifstream ::ifstream( const char*  
szName,      int nMode = ios::in,      int  
nProt = filebuf::openprot );
```

- 参数:
 - szName -- 文件名;
 - nMode -- 打开文件的方式。
 - ios::in -- 以读(输入)为目的打开。
 - ios::nocreate -- 仅打开一个已存在文件。
 - ios::binary -- 打开二进制文件。
 - nProt -- 指定所打开文件的保护方式。

文件流类的构造函数

```
ofstream::ofstream( const char*  
szName, int nMode = ios::out,  
int nProt = filebuf::openprot );
```

- 参数:
 - szName -- 文件名;
 - nMode -- 打开文件的方式。
 - ios::out -- 以写(输出)为目的打开文件。
 - ios::trunc -- 若文件存在, 则将文件长度截为0。
 - ios::binary -- 打开二进制文件。
 - ios::app -- 以追加方式打开。
 - ...
 - nProt -- 指定所打开文件的保护方式。

文件流类的构造函数

```
fstream::fstream( const char*  
szName,    int nMode,    int nProt =  
filebuf::openprot );
```

- 参数含义和用法与ofstream构造函数处相同

文件流类的open成员函数

ofstream::open

- `void open( const char* szName, int nMode = ios::out, int nProt = filebuf::openprot );`

ifstream::open

- `void open( const char* szName, int nMode = ios::in, int nProt = filebuf::openprot );`

fstream::open

- `void open( const char* szName, int nMode, int nProt = filebuf::openprot );`

使用插入运算符>>读文件

ifstream类由istream类所派生，而istream类中预定义了公有的运算符重载函数“operator >>”，所以，ifstream流（类对象）可以使用预定义的算符“>>”来对自定义磁盘文件进行“读”操作（允许通过派生类对象直接调用其基类的公有成员函数）；

使用提取运算符<<写文件

ofstream类由ostream类所派生，而ostream类中预定义了公有的运算符重载函数“operator <<”，所以，ofstream流（类对象）可以使用预定义的算符“<<”来对自定义磁盘文件进行“写”操作；

使用插入和提取运算符读写文件

fstream类由iostream所派生，iostream类由istream与ostream类共同派生，所以，fstream流（类对象）可以使用预定义的算符“>>”和“<<”来对自定义磁盘文件进行“读”与“写”操作

- 使用预定义的算符“<<”来进行“写”操作时，为了今后能正确读出，数据间要人为地添加分隔符（比如空格），这与用算符“>>”来进行“读”操作时遇空格或换行均结束一个数据相呼应。

【例10.4】

- 下述示例程序做了如下的3件事：1) 往文件写数据；2) 往文件尾部追加数据； 3) 从文件读出数据并显示在屏幕上

```
#include <fstream.h>
void main() {
    //1) 往文件写数据
    ofstream outfile1("myfile1.txt");
    //以“写”方式打开
    outfile1<<"Hello!...CHINA!  Nankai_University"<<endl;
    outfile1.close();
    //2) 往文件尾部追加数据
    outfile1.open("myfile1.txt", ios::app);
    //以“追加”方式打开
    int x=1212, y=6868;
    outfile1<<x<<" "<<y<<endl;
    //注意，数据间要人为添加分割符空格
    outfile1.close();
}
```

//3) 从文件读出数据并显示在屏幕上。

```
char str1[80], str2[80];  
int x2,y2;  
ifstream infile1("myfile1.txt");  
//以读方式打开  
infile1>>str1>>str2;  
//使用“>>”读(遇空格、换行均结束)  
infile1>>x2>>y2;  
infile1.close();  
cout<<"str1="<<str1<<endl;  
cout<<"str2="<<str2<<endl;  
cout<<"x2="<<x2<<endl;  
cout<<"y2="<<y2<<endl;  
}
```

程序执行后的显示结果如下:

```
str1=Hello!...CHINA!  
str2=Nankai_University  
x2=1212  
y2=6868
```

文件流类的成员函数

get()与put()

- 读写字符
- 多用于读写文本文件

read()与write()

- 读写二进制数据
- 多用于读写二进制文件

getline()

- 读字符串
- 多用于读文本文件

get()与put()读写文本文件

istream& get(**char& rch);**

- 功能：从自定义文件中读出1个字符放入引用rch中。

ostream& put(**char ch);**

- 功能：将字符ch写到自定义文件中。

注意，put实际上只是ostream类中定义的公有成员函数，当然通过其派生类ofstream的类对象也可以直接对它进行调用。同理，通过ifstream的类对象可以直接调用get。

【例10.5】

- 从键盘输入任一个字符串，通过put将其写到自定义磁盘文件“putgetfile.txt”中，而后再使用get从该文件中读出所写串并显示在屏幕上

```
#include <fstream.h>
#include <stdio.h>
void main() {
//键盘输入字符串，通过put将其写到自定义磁盘文件中
    char str[80];
    cout<<"Input string:"<<endl;
    gets(str);
    ofstream fout("putgetfile.txt");
    int i=0;
    while ( str[i] )
        fout.put(str[i++]);
    fout.close();
    cout<<"-----"<<endl;
```

```
//而后使用get从文件中读出该串显示在屏幕上
char ch;
ifstream fin("putgetfile.txt");
fin.get(ch);
while(!fin.eof()){
    //从头读到文件结束(当前符号非文件结束符时继续)
    //注: 成员函数eof()在读到文件结束时方返回true
    cout<<ch;
    fin.get(ch);
}
cout<<endl;
fin.close();
}
```

程序执行后的显示结果如下:

Input string:

File operating -- using 'put' and 'get', OK! 12345

File operating -- using 'put' and 'get', OK! 12345

【例10.6】

- 使用类成员函数get与put对指定文件进行拷贝。被拷贝的“源文件”以及拷贝到的“目的文件”的名字与路径均由命令行参数来提供
 - 程序运行: `copy mycopy.cpp res_1.cpp`
 - 程序执行结果(若提供的命令行参数为: `mycopy.cpp res_1.cpp`):
 - Copy file from 'mycopy.cpp' to 'res_1.cpp'
 - 程序执行结果(若提供的命令行参数个数不对):
 - ERROR! -- supplying 2 command-line argements ?!

```
#include <fstream.h>
#include <process.h>
void main(int argc, char* argv[]) {
    if(argc!=3) {
        //命令行参数个数不对时
        cout<<"ERROR ! -- supplying 2
command-line argements ?!"<<endl;
        exit (1);
    }
    cout<<"Copy file from '"<<argv[1]<<"' to
'"<<argv[2]<<"'"<<endl;
```

```
ifstream fin(argv[1]); //命令行参数1作为文件名
ofstream fout(argv[2]); //命令行参数2作为文件名

char ch;
fin.get(ch);
while (!fin.eof()) { //从头读到文件结束
    fout.put(ch);
    fin.get(ch);
}
fin.close();
fout.close();
}
```

read()和write()读写二进制文件

```
istream& read( char* pch, int nCount );
```

- 功能：从某个文件中读入nCount个字符放入pch缓冲区中（若读至文件结束尚不足nCount个字符时，也将立即结束本次读取过程）

```
ostream& write( const char* pch, int nCount );
```

- 功能：将pch缓冲区中的前nCount个字符写出到某个文件中
- 使用write与read进行数据读写时，不必要在数据间再额外“插入”分割符，这是因为它们都要求提供第二实参来指定读写长度。

【例10.7】

- 以下的示例程序先使用write往自定义二进制磁盘文件中写出如下3个“值”：字符串str的长度值Len(一个正整数)、字符串str本身、以及一个结构体的数据，而后再使用read读出这些“值”并将它们显示在屏幕上

```
#include <fstream.h>
#include <string.h>
void main() {
    char str[20]="Hello world!";
    struct stu{
        char name[20];
        int age;
        double score;
    } ss={"wu jun", 22, 91.5};
    cout<<"WRITE to 'wrt_read_file.bin'"<<endl;
```



```
ofstream fout("wrt_read_file.bin", ios::binary);  
//打开用于“写”的二进制磁盘文件  
int Len=strlen(str);  
fout.write( (char*)(&Len), sizeof(int) );  
fout.write(str, Len);  
//数据间无需分割符  
fout.write( (char*)(&ss), sizeof(ss) );  
fout.close();  
cout<<"-----"  
"<<endl;  
cout<<"-- READ it from 'wrt_read_file.bin' --"  
"<<endl;  
char str2[80];  
ifstream fin("wrt_read_file.bin", ios::binary);  
fin.read( (char*)(&Len), sizeof(int) );  
fin.read(str2, Len);
```

```
str2[Len]='\0';  
fin.read( (char*)(&ss), sizeof(ss) );  
cout<<"Len="<<Len<<endl;  
cout<<"str2="<<str2<<endl;  
cout<<"ss=>"<<ss.name<<" , "<<ss.age<<" , "<<ss.score<<en  
dl;  
fin.close();  
cout<<"-----"<<endl;  
}
```

程序执行后的显示结果如下:

```
WRITE to 'wrt_read_file.bin'
```

```
-----  
-- READ it from 'wrt_read_file.bin' --  
Len=12  
str2=Hello world!  
ss=>wu jun,22,91.5  
-----
```

getline()读文件

- 最常用格式为：

```
istream& getline(char* pch, int nCount,  
char delim = '\\n');
```

- 功能：从某个文件中读出一行（至多nCount个字符）放入pch缓冲区中，缺省行结束符为'\n'。
 - 如果nCount小于该行实际字符数，则读出nCount个字符
 - 如果nCount大于该行实际字符数，则读出该行全部字符

【例10.8】

- 程序实例: 读出 “getline_1.cpp”的各行并显示在屏幕上（如，可将本源程序存放在 “getline_1.cpp”文件中）

```
#include <fstream.h>
void main() {
    char line[81];
    ifstream infile("getline_1.cpp"); //打开文件用于读
    infile.getline(line, 80);
    //读出一行(至多80个字符)放入line中
    while(!infile.eof()) {
        //尚未读到文件结束则继续循环(处理)
        cout<<line<<endl; //显示在屏幕上
        infile.getline(line, 80); //再读一行
    }
    infile.close();
}
```

磁盘文件的输入与输出 ☐
文本文件与二进制文件 ☒
字符串流 ☐
其它输入输出函数 ☐

☐ 文本文件与二进制文件
☐ 按用户设置的文件类型进行读写
☐ 对数据文件的随机访问



插入与提取运算符重载



输入/输出格式控制



磁盘文件的输入与输出



文本文件与二进制文件



字符串流



其它输入输出函数

文本文件（.txt）

- 以文本形式存储
- 优点是具有较高的兼容性
- 缺点是存储一批纯数值信息时，要在数据之间人为地添加分割符，输入输出过程中，系统要对内外存的数据格式进行相应转换
- text文件不便于对数据实行随机访问。

二进制文件 (.bin)

- 以binary形式存储
- 优点是便于对数据实行随机访问（每一同类型数据所占磁盘空间的大小均相同，不必在数据之间人为地添加分割符），输入输出过程中，系统不对数据进行任何转换
- 缺点是兼容性低

设置文件的类型

由程序员决定将数据存储为text文件或者binary文件两种形式之一。

- 缺省打开方式时，默认为text文件形式。若欲使用binary文件形式，要将打开方式设为“ios::binary”。
- 通常将纯文本信息（如字符串）以text文件形式存储，而将数值信息以binary文件形式存储
- 通常使用read()与write()对二进制文件进行读写，但若非要使用它们对文本文件进行读写时，系统在write时有可能多写出了一些东西（如，回车换行符号等）。这样将导致read时产生错误

【例10.9】

- 从键盘读入多个结构数据(个数n由用户指定), 使用write将这些结构数据写出到某个自定义二进制磁盘文件中, 而后再使用read读出这些结构数据并进行处理(如, 求出n个score的平均值ave)

```
#include <fstream.h>
void main() {
    struct person {
        char name [20];
        int age;
        float score;
    } ss;
```

磁盘文件的输入与输出 ☐
文本文件与二进制文件 ☒
字符串流 ☐
其它输入输出函数 ☐

☐ 文本文件与二进制文件
☒ 按用户设置的文件类型进行读写
☐ 对数据文件的随机访问



```
int n;  
cin>>n; //个数n由用户指定  
ofstream fout("f01.bin", ios::binary); //打开二进制文件 (写)  
for(int i=0; i<n; i++) {  
    cin>>ss.name>>ss.age>>ss.score;  
    fout.write( (char *)(&ss), sizeof(ss)); //写到文件中  
}  
fout.close();  
ifstream fin("f01.bin", ios::binary); //打开二进制文件 (读)  
float ave=0;  
fin.read( (char *)(&ss), sizeof(ss)); //读入1个结构体数据放入ss  
while (!fin.eof()) { //未到文件末则继续  
    ave+=ss.score; //累加  
    fin.read( (char *)(&ss), sizeof(ss)); //再读数据放ss  
}  
fin.close();  
cout<<"n="<<n<<"    ave="<<ave/n<<endl;  
}
```

磁盘文件的输入与输出 ☐
文本文件与二进制文件 ☐
字符串流 ☐
其它输入输出函数 ☐

☐ 文本文件与二进制文件
☐ 按用户设置的文件类型进行读写
☒ 对数据文件的随机访问



对数据文件进行随机访问

使用类成员函数write与read，并配合使用类成员函数seekp和seekg，就可以对文件进行“随机性”（非顺序性）的读写操作

ostream::seekp()

```
ostream& seekp( long off, int dir =  
ios::beg );
```

- 功能：将“输出指针”的值置到一个新位置，使以后的输出从该新位置开始。新位置由参数off与dir之值确定：
 - 当dir之值为“ios::beg”时，新位置为：从文件首“后推” off字节处；
 - 当dir之值为“ios::cur”时，新位置为：从“输出指针”的当前位置“后推” off字节处；
 - 当dir之值为“ios::end”时，新位置为：从文件末“前推” off字节处。

istream::seekg()

```
istream& seekg( long off, int dir =  
ios::beg );
```

- 功能：将“读入指针”的值置到一个新位置，使以后的读入从该新位置开始。新位置由参数off与dir之值确定：
 - 当dir之值为“ios::beg”时，新位置为：从文件首“后推” off字节处；
 - 当dir之值为“ios::cur”时，新位置为：从“读入指针”的当前位置“后推” off字节处；
 - 当dir之值为“ios::end”时，新位置为：从文件末“前推” off字节处。

istream::tellg

```
long tellg( );
```

- 功能：获取“读入指针”的当前位置值。

ostream::tellp

```
long tellp( );
```

- 功能：获取“输出指针”的当前位置值。

【例10.10】

- 将a数组中存储的8个int型数据，
 - 首先通过运算符“<<”依次写出到text文件ft.txt之中
 - ✓ 注意各数据在文件中“长短”不一，且数据间必须加入分割符
 - 然后通过使用类成员函数write将这相同的8个int型数据依次写出到binary文件fb.bin之中
 - ✓ 注意各数据在文件中“长短”相同，且数据间不需要加入分割符
 - 通过使用无参的成员函数“tellp()”来获取当前已写出到各文件的位置信息，以确认每一数据在文件中所占的字节数。
 - ✓ ostream::tellp()之功能为：获取并返回“输出指针”的当前位置值（从文件首到当前位置的字节数）

磁盘文件的输入与输出 ☐
文本文件与二进制文件 ☐
字符串流 ☐
其它输入输出函数 ☐

☐ 文本文件与二进制文件
☐ 按用户设置的文件类型进行读写
☒ 对数据文件的随机访问



```
#include <fstream.h>
void main() {
    int a[8]={0,1,-1,1234567890};
    for(int i=4; i<8; i++)
        a[i]=876543210+i-4;
    //均由9位数字组成, 在text文件中所占字节数也为9
    ofstream ft("ft.txt");
    ofstream fb("fb.bin", ios::binary);
    for(i=0; i<8; i++) {
        ft<<a[i]<<" "; //数据间需要添加分割符
        fb.write((char*)(&a[i]), sizeof(a[i]));
        //数据间不需分割符
        cout<<"ft.tellp()="<<ft.tellp()<<" "; //当前ft文件位置
        cout<<"fb.tellp()="<<fb.tellp()<<endl; //当前fb文件位置
    }
    ft.close();
    fb.close();
}
```


磁盘文件的输入与输出 ☐
文本文件与二进制文件 ☐
字符串流 ☐
其它输入输出函数 ☐

☐ 文本文件与二进制文件
☐ 按用户设置的文件类型进行读写
☒ 对数据文件的随机访问



程序执行后的输出结果如下:

```
ft.tellp()=2,      fb.tellp()=4  
ft.tellp()=4,      fb.tellp()=8  
ft.tellp()=7,      fb.tellp()=12  
ft.tellp()=18,     fb.tellp()=16  
ft.tellp()=28,     fb.tellp()=20  
ft.tellp()=38,     fb.tellp()=24  
ft.tellp()=48,     fb.tellp()=28  
ft.tellp()=58,     fb.tellp()=32
```

【例10.11】

- 从键盘输入10个int型数，而后按输入的相反顺序输出它们。
 - 实现方法：使用binary文件，将数据存放在文件中，并使用随机访问方式读出

```
#include <fstream.h>
void main() {
    const int n=10;
    int x, i;
    ofstream fout("fdat.bin", ios::binary);
    cout<<"Input "<<n<<" integers:"<<endl;
    for(i=1; i<=n; i++){
        cin>>x;
        fout.write((char*)(&x), sizeof(int));
    }
    fout.close();
}
```

```
cout<<"---- The result ----"<<endl;
ifstream fin("fdat.bin", ios::binary);
for(i=n-1; i>=0; i--){
    fin.seekg(i*sizeof(int));
    fin.read((char*)(&x), sizeof(int));
    cout<<x<<" ";
}
fin.close();
cout<<endl;
}
```

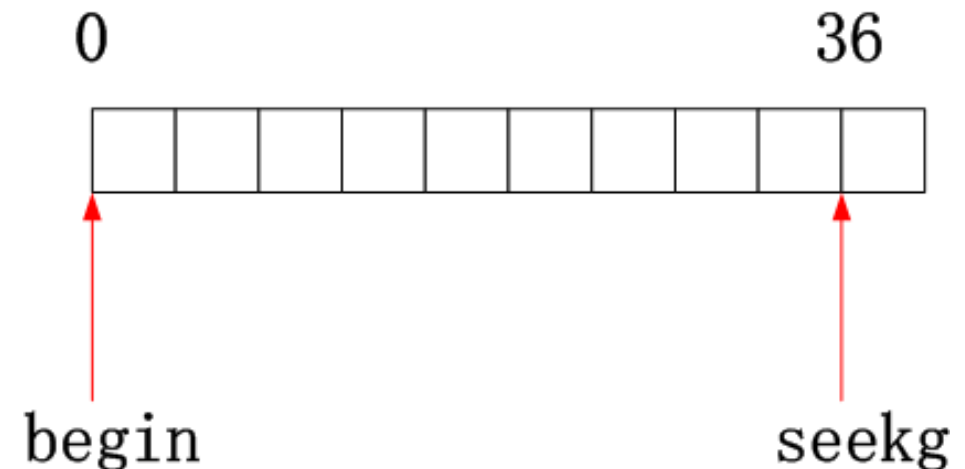
程序执行后的输出结果为:

Input 10 integers:

1 2 3 4 5 6 7 8 9 10

---- The result ----

10 9 8 7 6 5 4 3 2 1



【例10.12】

- 使用write将多个person类型的结构体数据，写出到某个自定义二进制磁盘文件的指定位置处，而后再使用read从另外指定的位置处读出某些结构体数据并显示在屏幕上。

一般程序处理模式:

```
struct person {  
    char name [20];  
    int age;  
    float score;  
}ss;
```

//说明一个结构体数组stu并赋初值，各结构体的分量num（编号）相互不同

```
person stu[10]={ {5, "zhou", 88.5}, {3, "sun", 89}, {7,  
    "zheng", 91.5}, {1, "zhao", 90.5}, {6, "wu", 94}, {2,  
    "qian", 91},{9, "feng", 87.5}, {4, "li", 84}, {8, "wang",  
    79}, {10, "chen", 90} };
```

```
int recnum;
```

磁盘文件的输入与输出 ☐
文本文件与二进制文件 ☐
字符串流 ☐
其它输入输出函数 ☐

☐ 文本文件与二进制文件
☐ 按用户设置的文件类型进行读写
☒ 对数据文件的随机访问



```
ofstream fout("f01.bin", ios::binary);
for( int i=0;i<10;i++ ){
    recnum = stu[i].num;// “纪录号”
    long offs= sizeof(ss)*(recnum-1);
    fout.seekp(offs);
    fout.write( (char *)(&ss), sizeof(ss));
}
fout.close();
ifstream fin("f01.bin", ios::binary);
cin>>recnum;
while ( recnum>0&&recnum<=10 ) {
    long offs= sizeof(ss)* (recnum-1);
    fin.seekg(offs);
    fin.read( (char *)(&ss), sizeof(ss));
    cout<<tmp.num<<" "<<tmp.name<<" "<<tmp.score<<endl;
}
fin.close();
}
```

磁盘文件的输入与输出 ☐
文本文件与二进制文件 ☐
字符串流 ☒
其它输入输出函数 ☐

☐ 字符串流类概述
☐ ostream类
☐ istream类
☐ stringstream类



插入与提取运算符重载



输入/输出格式控制



磁盘文件的输入与输出



文本文件与二进制文件



字符串流



其它输入输出函数

字符串流类

字符串流类对象并不对应于一个具体的物理设备，而是将内存中的字符数组看成是一个逻辑设备，并通过“借用”对文件进行操作的各种运算符和函数，最终完成上述所谓的信息转换工作

使用字符串流类时，必须包含头文件 `stringstream`

ostream类

通过ostream类的使用，可将不同类型的信息转换为字符串，并存放在（输出到）一个用户设定的字符数组中

- 构造函数ostream::ostream该类最常用的构造函数的一般格式为：

```
ostream(char* str, int n, int mode =  
ios::out );
```

- 函数ostream::pcount的使用格式为：

```
int pcount() const;
```

- 功能：返回一个数值，表示目前已经输出到字符串流即字符数组中的字符个数（字节数）

istream类

通过istream类的使用，则可将用户字符数组中的字符串取出（读入），而后反向转换为各种变量的内部形式

- 一参构造函数：`istream(char* str);`
 - 由参数str 指定了一个以'\0'为结束符的字符串（字符数组），它的“整体字符”将作为“输入源”。
- 二参构造函数：`istream(char* str, int n);`
 - 由参数str 指定字符数组，它将作为“输入源”，由第二参数n 指出仅使用str的前n个字符（而不是“整体字符”）
 - 二参构造函数时，并不要求str 中必须具有'\0'结束符号
 - 若n=0，则假定str 为一个以'\0'为结束符号的字符串（字符数组）

stringstream字符串流类

由iostream继承

支持string类型的字符串流类

- ostringstream向string写数据
- istreamstringstream从string读数据
- stringstream即可从string读数据，也可以像string写数据

主要成员：

- stringstream::str(): 返回字符串流保存的string
- stringstream::str(s): 将string类型的数据拷贝到stringstream对象

【例10.13】 用一个字符串包含浮点型数组的所有元素的文本表示，精度为4位，每行包含5个数，并在宽度为7个字符的输出域内右对齐。

```
#include <iomanip>
#include <sstream>
#include <string>
#include <vector>
using namespace std;

int main()
{
    vector<double> values;
    cout << "How many numbers do you want to enter? ";
    int num{};
    cin >> num;
```



```
for (int i{}; i < num; ++i) {  
    // Stream in all 'num' user inputs  
    double d{};  
    cin >> d;  
    values.push_back(d);  
}  
stringstream ss; // Create a new string stream  
for (int i{}; i < num; ++i) {  
    // Use it to compose the requested string  
    ss << setprecision(4) << setw(7) << right << values[i];  
    if ((i + 1) % 5 == 0)  
        ss << endl;  
}  
string s{ ss.str() };  
// Extract the resulting string using the str() function  
cout << s << endl;  
}
```

磁盘文件的输入与输出 ☐
文本文件与二进制文件 ☐
字符串流 ☒
其它输入输出函数 ☐

☐ 字符串流类概述
☐ ostream类
☐ istream类
☒ stringstream字符串流类



How many numbers do you want to enter? 7

1.23456

3.1415

1.4142

-5

17.0183

-25.1283

1000.456

1.235

3.142

1.414

-5

17.02

-25.13

1000

磁盘文件的输入与输出 ☐
文本文件与二进制文件 ☐
字符串流 ☐
其它输入输出函数 ☒

☐ 输入输出操作状态字
☐ 其它输入输出控制函数



插入与提取运算符重载



输入/输出格式控制



磁盘文件的输入与输出



文本文件与二进制文件



字符串流



其它输入输出函数

输入输出操作状态字

输入输出（I/O）操作状态字在类ios中定义，各位的状态由如下标志位（常量）进行描述：

- ios::goodbit=0x00
 - 流处于正常状态（没设置任何的状态标志位）
- ios::eofbit=0x01
 - 输入流结束（到达文件末尾）
- ios::failbit=0x02
 - I/O 操作失败（会使随后的操作也失败）
- ios::badbit=0x04
 - 失去了流缓冲区的完整性（流被破坏）

其它输入输出控制函数

good()

- `int good();`
- I/O 流正常（没设置任何的状态标志位）返回非0，否则返回0

eof()

- `int eof();`
- 到达文件末尾（状态字的eofbit 位被置1）则返回非0，否则返回0

fail()

- `int fail();`
- 流状态字的failbit、badbit 或hardfail 中任一个位被置1， 则返回非0（意味着随后的操作将失败）， 否则返回0

其它输入输出控制函数

bad()

- `int bad()` ;
- 流状态字的badbit 或hardfail 位中任一个被置1，则返回非0（严重错误，流被破坏），否则返回0

rdstate()

- `int rdstate()` ;
- 返回当前I/O操作状态字

operator!()

- `int operator!()` ;
- 与函数fail()功能相同

其它输入输出控制函数

clear()

- `void clear(int ef=0);`
- 无参调用可清除全部出错信息（将状态字的各位均清为0）；带参，可人工将某些状态标志位设置

第十章 结束

